

ISTM 637 Databricks Project Report

Emily Cendejas

NetID: Emily_cendejas

Catalog: istm637_emily_cendejas

Schema: oilgas

Project Overview

This project was built in Databricks using an oil and gas dataset organized in a simple star schema. The main tables I used were dim_well, dim_date, and fact_production, and I also created a well_forecast table for the predictive modeling part. Across the project, I used notebooks, Unity Catalog, Genie, a predictive model, Lakeflow, and an app connected to my SQL warehouse.

Data Model and Setup

The data model in my project is centered on production by well and date. dim_well stores well-level attributes like operator, basin, and well type, dim_date stores the calendar fields used for time analysis, and fact_production stores the daily oil, gas, and water production values. I used these tables throughout the project for SQL analysis, Genie testing, the forecasting notebook, and the app.

AI-Generated Comments and Corrections

For this part, I let the AI assistant write the first draft of the table and column descriptions, then went back and fixed the ones that didn’t really match the data. For example, it said well_type meant whether a well was oil, gas, or water, but in this project it’s actually about the wellbore shape, so I changed it to describe horizontal, vertical, and directional wells instead. I also updated the fact table comments to be clearer about units, like calling out that oil_bbl and water_bbl are barrels (bbl, 42 U.S. gallons) and gas_mcf is gas in thousand cubic feet (Mcf), so it’s obvious how to read the production numbers.

Genie Benchmark Questions and Results

Question	Correct?	Fix / Notes
Which basin produced the most oil overall?	Yes	Worked well after I added a trusted query and clear comments on oil_bbl and basin.
Show monthly oil production trend for 2024.	Yes	No change needed. Genie used the date dimension for a good line chart and narrative.

What is the average water cut for wells in Permian?	Yes	No change needed. Answer matched the definition using <code>water_bbl / (oil_bbl + water_bbl)</code> .
Which operator has the most producing wells?	Yes	No change needed. Metadata on status and operator helped Genie filter to producing wells.
List the top 5 wells by total gas production.	Yes	No change needed; trusted query that sums <code>gas_mcf</code> by <code>well_id</code> produced the correct table.

Overall, Genie performed better after I cleaned up the metadata, added better comments, and created trusted queries for the main business questions. The biggest improvement came from making the table and column meanings more specific so the assistant would not guess what fields like `well_type` meant.

Predictive Model

For the predictive modeling part, I used my Databricks notebook to train a model using the production data and generate forecasts that I later stored in the `well_forecast` table. The goal of this section was to predict future oil production based on the historical production patterns in my dataset.

My model results were:

- **Target variable:** oil production (bbl/day)
- **MAE:** 33.5 bbl/day
- **RMSE:** 70.0 bbl/day
- **R²:** 0.933

These results show that the model fit the data pretty well, since the R² value was 0.933 and the prediction error stayed fairly low compared with the range of production values in the dataset. For this project, that was good enough to use the forecast output in the app as a precomputed prediction instead of trying to calculate forecasts live.

App Data Source Explanation

The app pulls everything from the Unity Catalog tables I built earlier in the project. The well dropdown is driven by the `dim_well` table using the `well_id` field, and the history chart comes from `fact_production` joined to `dim_date` to roll daily `oil_bbl` up to months. The

forecast line isn't calculated live in the app. It reads the precomputed predictions from my `well_forecast` table that I generated in the model notebook. All of these queries run on my serverless SQL warehouse, with the app's identity granted `USE CATALOG`, `USE SCHEMA`, and `SELECT` on my `istm637_emily_cendejas.oilgas` objects.

OpenSharing

For the OpenSharing part, I created a share called `dim_well_share` and added the `dim_well` table from my `oilgas` schema. I was able to set up the share and add a Databricks recipient, but the free edition limits meant I could not fully test the share end to end after sending it.

If the recipient was not using Databricks, they would consume the share through the open Delta/OpenSharing protocol instead of logging into a Databricks workspace. As the provider, I would generate an external recipient and send them a profile file. That file holds the sharing server URL, a token, and pointers to the tables I shared. The recipient downloads this file and uses an open-source client, such as the `delta-sharing` Python library, to read the data.

If they were using SQL, they would use this:

```
SELECT *  
FROM  
delta_sharing.`dim_well_share_credentials.share#dim_well_share.oilgas.dim_well`  
LIMIT 10;
```

The main point is that the recipient would just need the credential file plus an open client. They would not log into my Databricks workspace, but they could still query the `dim_well` table from their own tools.

Lakeflow Ingest Pipeline

For the Lakeflow part, I created and committed my ingest pipeline SQL file as part of the repository submission. This part of the project shows how Databricks can manage ingestion as a pipeline instead of only using notebook-based loading steps. The pipeline is included in my repo as `ISTM637_Lakeflow_Ingest_Pipeline.sql`, and the `screenshots` folder includes evidence of the setup and execution.

Screenshots Included

My screenshots pdf includes images showing evidence for Parts 1–8 of the project, including Git workflow screenshots, table creation and loading, Genie testing, model results, the app, OpenSharing setup, and Lakeflow pipeline setup.

Repository Contents

My GitHub repository submission includes the following required files:

- ISTM637_Databricks_Project_Starter.ipynb
- ISTM637_Predictive_Model_Notebook.ipynb
- ISTM637_Lakeflow_Ingest_Pipeline.sql
- REPORT.pdf
- Screenshots.pdf
- README.md

It also includes at least two commits to show the Git check-in / check-out workflow required in Part 1.